Cryptography

Introduction

Clicker quiz

- Using the English-language shift cipher, what is the encryption of “good” using the key ‘b’?
  1. XYYD
  2. HPPE
  3. QRST
  4. GNNE

Clicker quiz

- Using the English-language shift cipher, which of the following plaintexts could correspond to ciphertext AZC?
  1. can
  2. bad
  3. dog
  4. run

Byte-wise shift cipher

- Work with an alphabet of *bytes* rather than (English, lowercase) *letters*
  - Works natively for arbitrary data!

- Use XOR instead of modular addition
  - Essential properties still hold

ASCII

- Characters (often) represented in ASCII
  - 1 byte/char = 2 hex digits/char

Hex	Dec	Char		Hex	Dec	Char	Hex	Dec	Char	Hex	Dec	Char
0x00	0	NULL	null	0x20	32	Space	0x40	64	@	0x60	96	`
0x01	1	SOH	Start of heading	0x21	33	!	0x41	65	A	0x61	97	a
0x02	2	STX	Start of text	0x22	34	"	0x42	66	B	0x62	98	b
0x03	3	ETX	End of text	0x23	35	#	0x43	67	C	0x63	99	c
0x04	4	EOT	End of transmission	0x24	36	$	0x44	68	D	0x64	100	d
0x05	5	ENQ	Enquiry	0x25	37	%	0x45	69	E	0x65	101	e
0x06	6	ACK	Acknowledge	0x26	38	&	0x46	70	F	0x66	102	f
0x07	7	BELL	Bell	0x27	39	'	0x47	71	G	0x67	103	g
0x08	8	BS	Backspace	0x28	40	(	0x48	72	H	0x68	104	h
0x09	9	TAB	Horizontal tab	0x29	41	)	0x49	73	I	0x69	105	i
0x0A	10	LF	New line	0x2A	42	*	0x4A	74	J	0x6A	106	j
0x0B	11	VT	Vertical tab	0x2B	43	+	0x4B	75	K	0x6B	107	k
0x0C	12	FF	Form Feed	0x2C	44	,	0x4C	76	L	0x6C	108	l
0x0D	13	CR	Carriage return	0x2D	45	-	0x4D	77	M	0x6D	109	m
0x0E	14	SO	Shift out	0x2E	46	.	0x4E	78	N	0x6E	110	n
0x0F	15	SI	Shift in	0x2F	47	/	0x4F	79	O	0x6F	111	o
0x10	16	DLE	Data link escape	0x30	48	0	0x50	80	P	0x70	112	p
0x11	17	DC1	Device control 1	0x31	49	1	0x51	81	Q	0x71	113	q
0x12	18	DC2	Device control 2	0x32	50	2	0x52	82	R	0x72	114	r
0x13	19	DC3	Device control 3	0x33	51	3	0x53	83	S	0x73	115	s
0x14	20	DC4	Device control 4	0x34	52	4	0x54	84	T	0x74	116	t
0x15	21	NAK	Negative ack	0x35	53	5	0x55	85	U	0x75	117	u
0x16	22	SYN	Synchronous idle	0x36	54	6	0x56	86	V	0x76	118	v
0x17	23	ETB	End transmission block	0x37	55	7	0x57	87	W	0x77	119	w
0x18	24	CAN	Cancel	0x38	56	8	0x58	88	X	0x78	120	x
0x19	25	EM	End of medium	0x39	57	9	0x59	89	Y	0x79	121	y
0x1A	26	SUB	Substitute	0x3A	58	:	0x5A	90	Z	0x7A	122	z
0x1B	27	FSC	Escape	0x3B	59	;	0x5B	91	[	0x7B	123	{
0x1C	28	FS	File separator	0x3C	60	<	0x5C	92	\	0x7C	124	|
0x1D	29	GS	Group separator	0x3D	61	=	0x5D	93	]	0x7D	125	}
0x1E	30	RS	Record separator	0x3E	62	>	0x5E	94	^	0x7E	126	~
0x1F	31	US	Unit separator	0x3F	63	?	0x5F	95	_	0x7F	127	DEL

Source: http://benborowiec.com/2011/07/23/better-ascii-table/

Useful observations

- Only 128 valid ASCII chars (128 bytes invalid)
- Only 0x20-0x7E printable
- 0x41-0x7a includes upper/lowercase letters
  - Uppercase letters begin with 0x4 or 0x5
  - Lowercase letters begin with 0x6 or 0x7

Byte-wise shift cipher

- $\mathcal{M}$ = {strings of bytes}
- Gen: choose uniform $k \in \mathcal{K}$ = {0x00, …, 0xFF}
  - 256 possible keys
- $Enc_k(m_1 \ldots m_t)$: output $c_1 \ldots c_t$, where
  $$c_i := m_i \oplus k$$
- $Dec_k(c_1 \ldots c_t)$: output $m_1 \ldots m_t$, where
  $$m_i := c_i \oplus k$$

- Verify that correctness holds…

Code for byte-wise shift cipher

```
// read key from key.txt (hex) and message from ptext.txt (ASCII);
// output ciphertext to ctext.txt (hex)
#include <stdio.h>

main(){
  FILE *keyfile, *pfile, *cfile;
  int i;
  unsigned char key, ch;

  keyfile = fopen("key.txt", "r"), pfile = fopen("ptext.txt", "r"), cfile = fopen("ctext.txt", "w");

  if (fscanf(keyfile, "%2hhX", &key)==EOF) printf("Error reading key.\n");

  for (i=0; ; i++){
    if (fscanf(pfile, "%c", &ch)==EOF) break;
    fprintf(cfile, "%02X", ch^key);
   }

  fclose(keyfile), fclose(pfile), fclose(cfile);
}
```

Is this scheme secure?

- No -- only 256 possible keys!
  - Given a ciphertext, try decrypting with every possible key
  - If ciphertext is long enough, only one plaintext will "make sense"
- Can further optimize
  - First nibble of plaintext likely 0x4, 0x5, 0x6, 0x7 (assuming letters only)
  - Under plausible assumptions, can reduce exhaustive search to 26 keys (how?)

Sufficient key space principle

- The key space must be large enough to make exhaustive-search attacks impractical
  - How large do you think that is?

- Technical note (more next lecture): this is only true when the ciphertext is sufficiently long

The Vigenère cipher

- The key is now a *string*, not just a character
- To encrypt, shift each character in the plaintext by the amount dictated by the next character of the key
  - Wrap around in the key as needed
- Decryption just reverses the process

```
tellhimaboutme
cafecafecafeca
veqpjiredozxoe
```

The Vigenère cipher

- Size of key space?
  - If keys are 14-character strings over the English alphabet, then key space has size $26^{14} \approx 2^{66}$
  - If variable length keys, even more…
  - Brute-force search infeasible
- Is the Vigenère cipher secure?
- (Believed secure for many years…)

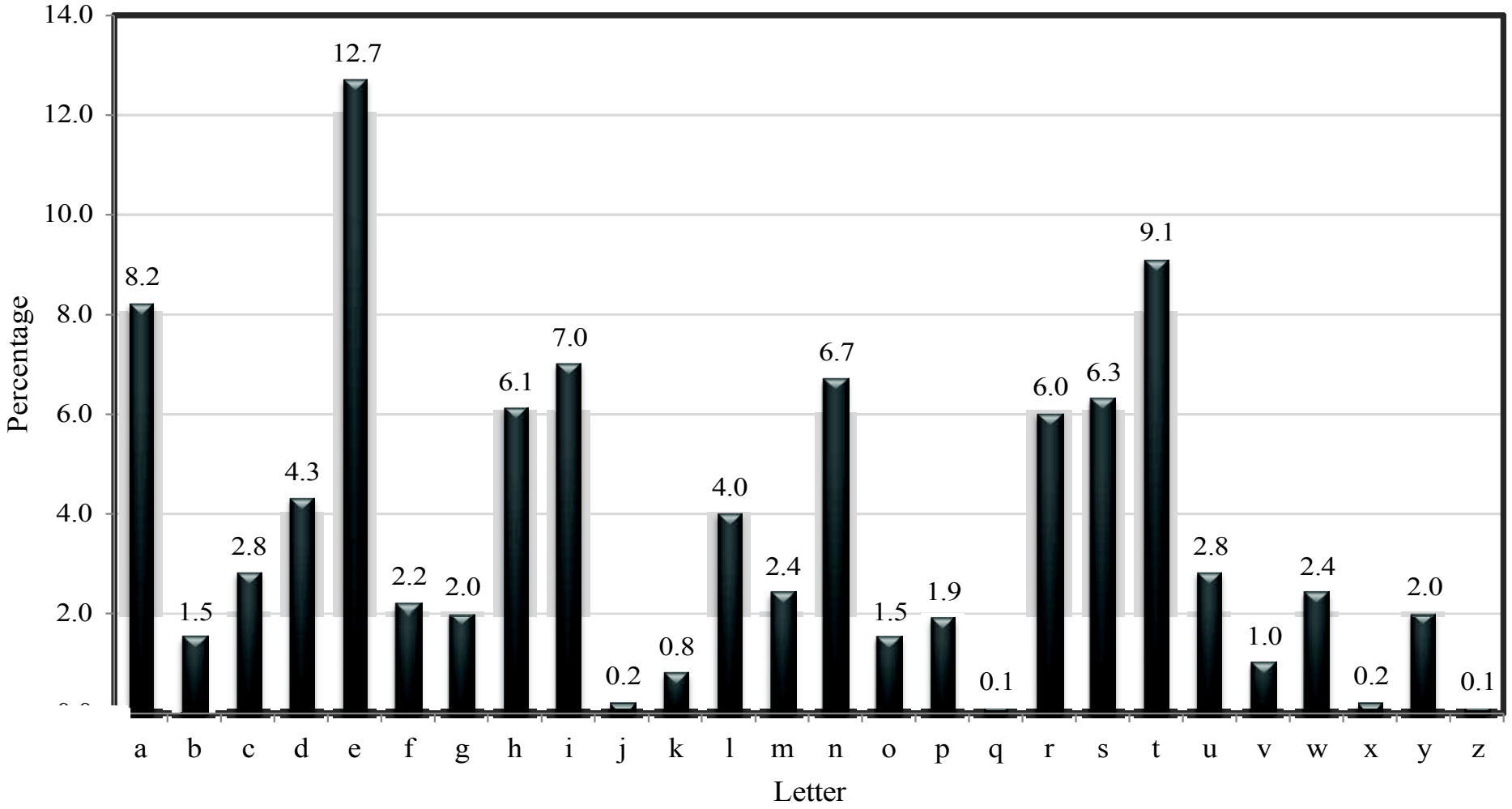
Attacking the Vigenère cipher

- (Assume a 14-character key)
- Observation: every 14th character is "encrypted" using the same shift
- Looki
  (almc
  encry

```
veqpjiredozxoeualpcmsdjqu
iqndnossoscdcusoakjqmxpqr
  hyycjqoqqodhjcciowieii
```

  - Though a direct brute-force attack doesn't work…
  - Why not?

Using plaintext letter frequencies



Attacking the Vigenère cipher

- Look at every 14th character of the ciphertext, starting with the first
  - Call this a “stream”
- Let α be the most common character appearing in this stream
- Most likely, α corresponds to the most common plaintext character (i.e., ‘e’)
  - Guess that the first character of the key is α - ’e’
- Repeat for all other positions

- This is somewhat haphazard…and does not use all the information available

A better attack

- Let p_i ($0 \le i \le 25$) denote the frequency of the i^{th} English letter in normal English plaintext
  - One can compute that $\sum_i p_i^2 \approx 0.065$
- Let q_i denote the observed frequency of the i^{th} letter in a given stream of the ciphertext
- If the shift for that stream is j, expect $q_{i+j} \approx p_i$ for all i
  - So expect $\sum_i p_i q_{i+j} \approx 0.065$
- Test for every value of j to find the right one
  - Repeat for each stream

Finding the key length

- The previous attack assumes we know the key length
  - What if we don't?
- Note: can always try the previous attack for all possible key lengths
  - # of key lengths << # keys
- Possible to do better!

Finding the key length

- When using the correct key length, the ciphertext frequencies $\{q_i\}$ of a stream will be *shifted versions* of the $\{p_i\}$
  - So $\Sigma\, q_i^2 \approx \Sigma\, p_i^2 \approx 0.065$
- When using an incorrect key length, expect (heuristically) that ciphertext letters are uniform
  - So $\Sigma\, q_i^2 \approx \Sigma\, (1/26)^2 = 1/26 = 0.038$

- In fact, good enough to find the key length N that maximizes $\Sigma\, q_i^2$
  - Can verify by looking at other streams…

Byte-wise Vigenère cipher

- The key is a string of bytes
- The plaintext is a string of bytes
- To encrypt, XOR each character in the plaintext with the next character of the key
  - Wrap around in the key as needed
- Decryption just reverses the process

Example

- Say plaintext is “Hello!” and key is 0xA1 2F
- “Hello!” = 0x48 65 6C 6C 6F 21
- XOR with 0xA1 2F A1 2F A1 2F
- 0x48 ⊕ 0xA1
  - 0100 1000 ⊕ 1010 0001 = 1110 1001 = 0xE9

- Ciphertext: 0xE9 4A CD 43 CE 0E

Attacking the (variant) Vigenère cipher

- As before, two steps:
  - Determine the key length
  - Determine each byte of the key

- Let p_i (for $0 \leq i \leq 255$) be the frequency of **byte** i in normal English (ASCII) plaintext
  - I.e., $p_i = 0$ for $i < 32$ or $i > 127$
  - I.e., p_{97} = frequency of 'a'

- If $\{p_i\}$ are known, use same principles as before…
  - What if they are not known?

Determining the key length

- If the key length is N, every N^{th} character of plaintext is encrypted using the same "shift"
  - If we take every N^{th} character and calculate frequencies, we get the $\{p_i\}$ in permuted order
  - If we take every M^{th} character (M not a multiple of N) and calculate frequencies, we get something close to uniform
  - We don't need to know the $\{p_i\}$ to distinguish these two!

Determining the key length

- For some candidate key length, tabulate q_0, …, q_{255} for first stream and compute $\Sigma\, q_i^2$
  - If close to uniform, $\Sigma\, q_i^2 \approx 256 \cdot (1/256)^2 = 1/256$
  - If a permutation of p_i, then $\Sigma\, q_i^2 \approx \Sigma\, p_i^2$
    - Key point: will be much larger than 1/256
- So: compute $\Sigma\, q_i^2$ for each possible key length, and look for maximum value
  - Correct key length N should yield a large value for all N streams

Determining the i^{th} byte of the key

- Assume the key length N is known
- Look at i^{th} ciphertext stream
  - As before, all bytes in this stream were generated by XORing plaintext with the same byte of the key
- Try decrypting the stream using every possible byte value B
  - Get a candidate plaintext stream for each value

Determining the i^{th} byte of the key

- When the guess B is correct:
  - All bytes in the plaintext stream will be between 32 and 126
  - Frequency of space character should be high
  - Frequencies of lowercase letters (as a fraction of all lowercase letters) should be close to known English-letter frequencies
    - Tabulate observed letter frequencies $q'_0, \ldots, q'_{25}$ (as fraction of all lowercase letters) in the candidate plaintext
    - Should find $\Sigma\, q'_i\, p'_i \approx \Sigma\, p'^2_i \approx 0.065$, where p'_i corresponds to English-letter frequencies
    - In practice, take B that maximizes $\Sigma\, q'_i\, p'_i$, subject to caveats above (and possibly others)

Attack time?

- Say the key length is between 1 and L
- Determining the key length: $\approx 256\ L$
- Determining all bytes of the key: $< 256^2\ L$

- Brute-force key search: $\approx 256^L$

The attack in practice

- Attack is more reliable as the ciphertext length grows larger

- Attack still works for short(er) ciphertexts, but more “tweaking” and manual involvement may be needed

First programming assignment

- Project 1 : 09.09.2025
- A number of people have worked out how to solve polyalphabetic ciphers:
  - Womaniser Giacomo Casanova
  - Charles Babbage
  - First published solution was in 1863 by Friedrich Kasiski, a Prussian infantry officer
    - He noticed that given a long enough piece of ciphertext, repeated patterns will appear at multiples of the keyword length.

Important

- Designing secure ciphers is hard.
- The Vigenere cipher remained unbroken for a long time.
- Far more complex schemes have also been used.
- A complex scheme is not necessarily secure, and all historical schemes have been broken.

So far…

- “Heuristic” constructions; construct, break, repeat, …
- Can we *prove* that some encryption scheme is secure?
- First need to *define* what we mean by “secure” in the first place…

Historically…

- Cryptography was an *art*
  - Heuristic design and analysis
- This isn't very satisfying
  - How do we know when a scheme is secure?

Modern cryptography

- In the late ‘70s and early ‘80s, cryptography began to develop into more of a *science*

- Based on three principles that underpin most crypto work today

Definitions

- Cryptography refers to the science and art of designing ciphers
- Cryptanalysis to the science and art of breaking them
- Cryptology, often shortened to just crypto, is the study of both. The input to an encryption process is commonly called the plaintext or cleartext, and the output the ciphertext.

Core principles of modern crypto

- Principle 1 – Formal Definitions
  - formal definitions of security are essential for the proper design, study, evaluation, and usage of cryptographic primitives.
  - *If you don't understand what you want to achieve, how can you possibly know when (or if) you have achieved it?*
  - Formal definitions provide such understanding by giving a clear description of what threats are in scope and what security guarantees are desired.
  - formalize what is required before the design process begins
- Definitions enable a meaningful comparison of schemes : a scheme satisfying a weaker definition may be more efficient than another scheme satisfying a stronger definition; with precise definitions we can properly evaluate the trade-offs between the two schemes.

Core principles of modern crypto

- Principle 1 : formal definitions
  - In general, a security definition has two components: a security guarantee and a threat model.
  - The security guarantee defines what the scheme is intended to prevent the attacker from doing, while the threat model describes the power of the adversary, i.e., what actions the attacker is assumed able to carry out.

Core principles of modern crypto

- Principle 2 – Precise Assumptions
  - Most modern cryptographic constructions cannot be proven secure unconditionally
  - Proofs would require resolving questions in the theory of computational complexity that seem far from being answered today.
  - Modern cryptography requires any such assumptions to be made explicit and mathematically precise.

Core principles of modern crypto

- Principle 3: Proofs of Security
  - The two principles described before allow us to achieve our goal of providing a rigorous proof that a construction satisfies a given definition under certain specified assumption
  - Proofs are especially important in the context of cryptography where there is an attacker who is actively trying to “break” some scheme.
  - Proofs of security give an iron-clad guarantee—relative to the definition and assumptions—that no attacker will succeed
  - Never: taking an unprincipled or heuristic approach to the problem.
  - Without a proof that no adversary with the specified resources can break some scheme, we are left only with our intuition that this is the case.
  - Experience has shown that intuition in cryptography and computer security is disastrous.

TSE publica edital do Teste Público de Segurança dos Sistemas Eleitorais 2025

Iniciativa reforça o compromisso da Justiça com a transparência e a segurança das Eleições Gerais 2026

27/06/2025 17:56 - Atualizado em 27/06/2025 18:47

Proofs of security give an iron-clad guarantee—relative to the definition and assumptions—that no attacker will succeed

Conclusions

- The approach taken by modern cryptography has revolutionized the field : provide confidence in the security of cryptographic schemes deployed in the real world.
- A proof of security is always relative to the definition being considered and the assumption(s) being used.
- If the security guarantee does not match what is needed, or the threat model does not capture the adversary's true abilities, then the proof may be irrelevant.
- Similarly, if the assumption that is relied upon turns out to be false, then the proof of security is meaningless.
- Provable security of a scheme does not necessarily imply security of that scheme in the real world.